

Technical Test : Front End Development

Updated March 2022

What we are looking for

With this test, we are looking to assess your programming and Javascript skills and your creativity as a developer. This test is open-ended on purpose, there is no perfect answer. We will also look at your process during the completion of this test, including the use of git, the documentation you provide and the creativity you demonstrated. As a baseline of course, we want to see that you're very comfortable with the foundations of web development: HTML, CSS, and JS.

For more senior applicants, it's important for us that you understand what's behind front-end frameworks and demonstrate strong development practices. Attention to detail is important, along the same lines it's expected that the ultimate deliverable runs smoothly without errors. This test is also an opportunity to show what tools you're comfortable with, but please don't try to learn the tools you're not comfortable with just to "cover ground."

Deliverables

Your test must be delivered as a public Github repository (you'll need a Github account for this). If for whatever reason you are unable to create a Github repository, you can also send us your test in a zip file, if it includes the .git folder and the git commits history. **Note: please do NOT use the word Streamroot in your repository name or project** (it is a registered trademark). If describing the use-case, we encourage phrasing like "distributed delivery" or "peer accelerated streaming."

Format

You must create a react application to implement 3 features, to handle the backend that is shared with you in this document.

The challenge is to implement as much as possible, with your best code quality.

FAQ

- ***Is there any required tech to use?***

Well, yes!

You must use TypeScript and React to complete the tasks in the tests since our dashboard has been written with these technologies. Proficiency with other libraries is great as it provides a perspective outside of React, however you would be using React + TS in your day to day so we must start there.

We would also like to see how you use styled components.

- ***What if I use a specific build system and / or libraries?***

Be sure to include the necessary instructions to build and run your application in a Readme.md or Instructions.md file. We should be able to take these instructions and successfully build the project from scratch. You should only assume that node + npm/yarn is installed, but if a certain version is needed, please note that.

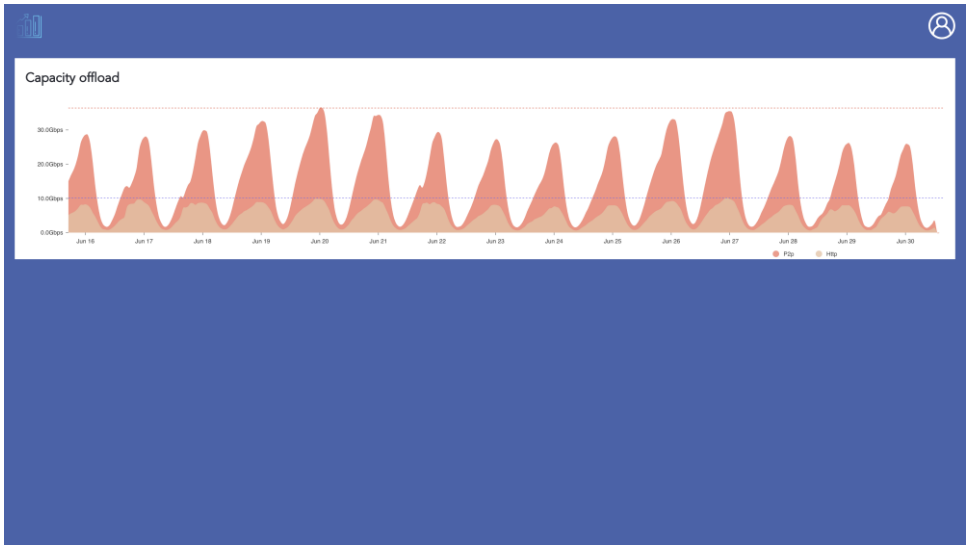
The challenge

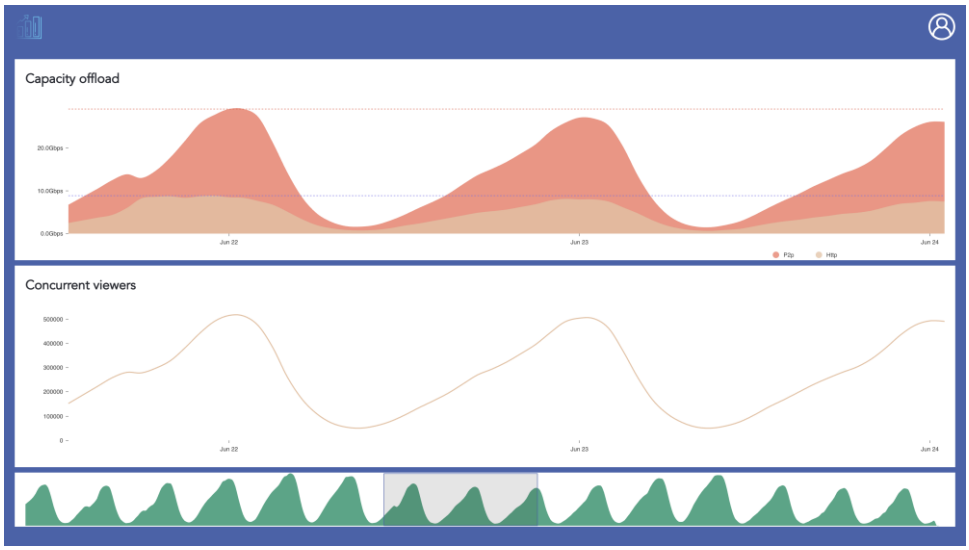
Description

The challenge is to implement some features on top of the backend application that we shared with you.

You will find two screen below. The first one is the application with only the capacity offload graph. The second one is the result that you must provide at the end of this test.

You are free to implement the design you want for these two new features. The implemented design you see on the two screen are not mandatory.





Feature 1

Description

The goal of this feature Is to implement the login feature. And the logout feature.
You will find code in the mock backend to implement those two features on your new frontend.

Please find bellow examples of what it can look like.



Feature 2

Description

The goal of this feature is to implement the two graphs called “Capacity offload” and “concurrent viewers” on the second screen above.

Capacity offload graph data to display:

- P2P (graph)
- Max P2P (stroke)
- Http (graph)
- Max Http (stroke)
- Tooltip:
    - P2P
    - Max P2P (stroke)
    - Http
    - Max Http (stroke)
    - Total
    - Spike reduction

Concurrent viewer graph data to display:

- Audience (graph)
- Tooltip:
    - Audience

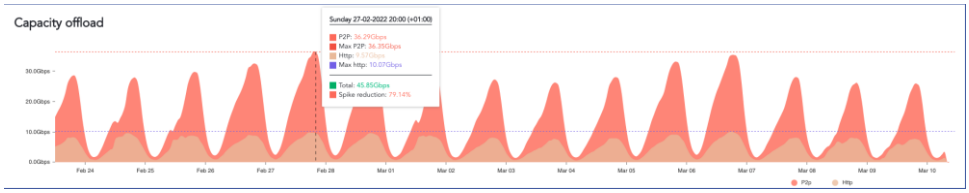
On the first graph called “Capacity offload” You must also implement a stroke that represent the max P2P and the max Http.

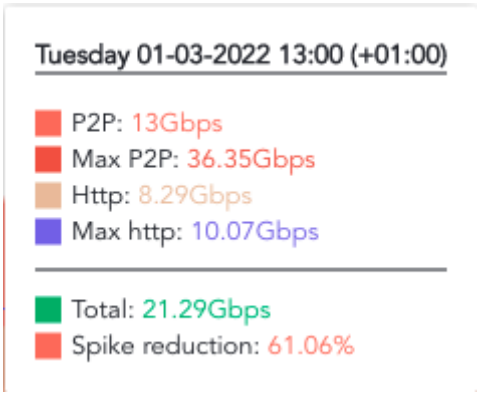
You will need also to implement the tooltip to display data in it. Take a look at the tooltip in the first graph to help you.

All the resources are provided by the backend.

You will have to dig a bit this backend to find how it works and what route you will need to implement to get the data.

The tool tip should look something like this screenshot bellow.


Capacity offload
Sunday 27-02-2022 20:00 (+01:00)
P2P: 36.29Gbps
Max P2P: 36.35Gbps
Http: 8.37Gbps
Max http: 10.07Gbps
Total: 45.85Gbps
Spike reduction: 79.14%
50.0Gbps
20.0Gbps
10.0Gbps
0.0Gbps
Feb 24
Feb 25
Feb 26
Feb 27
Feb 28
Mar 01
Mar 02
Mar 03
Mar 04
Mar 05
Mar 06
Mar 07
Mar 08
Mar 09
Mar 10
P2p
Http


Tuesday 01-03-2022 13:00 (+01:00)
P2P: 13Gbps
Max P2P: 36.35Gbps
Http: 8.29Gbps
Max http: 10.07Gbps
Total: 21.29Gbps
Spike reduction: 61.06%

Feature 3

Description

This feature implements the 3rd graph you can see on Figure 2.

You will need to develop a component that is called a brush graph.

This component lets you brush the graph with the mouse and the area that has been brushed will then represent the new interval of time you will display on the two other graphs.

The action flow is the following:

- User clicks on the brush graph to start selecting the new timeframe
- User maintains pressure on the mouse until she selects the desired timeframe, and then stops the click/brush.
- The 2 graphs above (Capacity offload and Concurrent users) are updated with a new timeframe corresponding to the one selected in the brush graph.

This feature is a great way to let the user “zoom” and “un-zoom” the original timeseries graphs that are initially loaded on the page.

Please take a look at following screenshots to help you:

Start the brush



During the brush



Resources

Mock backend:

1) We have provided a mock backend for you to make requests to. You'll have to understand and use this mock backend to fetch the data you need to complete your test project. But you're free to use, to delete or to add what you want in this mock backend or even create a new one! If you modify, add or remove functionality, please don't forget to send it to us!
2) Link to Mock Backend
3) To run it you'll need to have node and npm installed on your computer and then follow the README.md.
4) In this backend you have different data stored in JSON files, you'll have to use what you think useful thanks to the routes provided in the mock-backend.js file.
5) A README.md help you to understand the backend but you'll probably have to dig into the code to fully understand it.
6)

Data provided :

1) Bandwidth: data for the cdn vs p2p graph (date timestamp, bandwidth value).
2) Audience: data for the concurrent viewers graph (date timestamp, audience value).
3) Traffic: data to list the streams, you can use this data to present streams information

Design Resources

If you'd like to use some of our basic style, please feel free! This is not required.

Lumen design

To develop its product Lumen has built a design framework called Chi, that regroup a css layer and a javascript layer. We encourage you to use at least the css part.

You can find the documentation at those addresses.

- Github repository
  - https://github.com/CenturyLink/Chi
- Documentation
  - https://assets.ctl.io/chi/4.1.1/getting-started/

Good luck!

